

Assignment 3

Artificial Neural Networks for Classification

© Industrial Analytics Group
 Dept. Civil and Industrial Engineering, Uppsala University
 November 4, 2025

Contents

1	Introduction	2
2	Theory	2
2.1	The Softmax layer	2
2.1.1	Problem 1: Softmax layer example	3
2.1.2	Problem 2: Softmax layer properties	3
2.2	Cross Entropy for classification	3
2.2.1	Problem 3: Average Cross Entropy	3
2.3	The Receiver Operating Characteristic (ROC) curve	3
2.3.1	Problem 4: ROC curves	5
2.4	Performance measures for more than two classes	5
3	Task A: Two-class Signal Classification by FANNs	6
3.1	Overall description	6
3.2	Accessing provided data and code	6
3.3	Problems	7
4	Task B: Multi-class Image Classification of FashionMNIST by FANNs	10
4.1	The images	10
4.2	The labels	11
4.3	Importing the dataset	11
4.4	The actual tasks to perform	11
5	Task C: Convolutional ANNs	13
5.1	Problem 1: CNN Theory	13
5.2	Problem 2: Image Classification using a CNN	14
5.3	Problem 3: Performance Estimation of a CNN	15
6	DEADLINE AND EXAMINATION	16
7	Extra Study Material, Tips, Etc	17
7.1	PyTorch Help, Troubleshooting and Guidance	17
7.2	Neural Networks, what they actually do and how they do it	17
7.3	Gradient Decent and Stochastic Gradient Decent	17
7.4	Backpropagation to achieve auto-differentiation	17
7.5	Convolutional Neural Networks	17

1 Introduction

This assignment assumes you have already from the lectures and readings gained a reasonable understanding about what is a feedforward artificial neural network (FANN) and the surrounding theory and terminology for regression problems. The overarching aims here are to learn about:

- how softmax layers work in practice by transforming an input vector into an output vector with values on the unit interval $[0, 1]$ that always sum to one.
- the Cross Entropy cost function for use in classification problems
- how common data like signals and images are represented as one/multi-dimensional numerical arrays and tensors in order to be useful for FANN training.
- different performance measures commonly used to report results from 2-class classification problems such as probability of detection and probability of false alarm, including alternative terminology (such as true positive rate and false positive rate).
- different performance measures commonly used to report results from classification problems with more than two classes ("multi-class classification") such as accuracy and confusion matrices.
- how to solve large-scale classification problems using FANNs in PyTorch.
- how to solve large-scale real world classification problems using Convolutional Neural Networks (CNNs) in PyTorch.

2 Theory

2.1 The Softmax layer

When designing and training a FANN for classification, the standard operating procedure is to have a so-called *Softmax* layer as the final one of the network, instead of a conventional neuron layer where each neuron works independently of the others. Consider the case when there are K classes and the the Softmax layer consists of K different outputs nodes. Assume that for a particular network input $\mathbf{x}_n$, node k of the Softmax layer gets the input v_{kn} from the previous layer in the FANN. Then the output y_k from this node can be written as

$$y_{kn} = \frac{e^{v_{kn}}}{\sum_{k=1}^K e^{v_{kn}}}$$

For an illustration in the special case of $K = 4$, see Fig.1

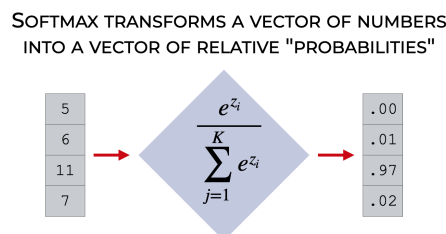


Figure 1: Illustration of the transformation made by the softmax function when $K = 4$.

2.1.1 Problem 1: Softmax layer example

Assume you have a 3-class problem and that a network input $\mathbf{x}_n$ results in the following inputs to the Softmax layer: $v_{1n} = 1$, $v_{2n} = 3$, $v_{3n} = 9$.

- (a) Determine the corresponding three outputs y_{kn} from the Softmax layer.
- (b) Based on the results in (a) and what is shown in Fig.1, try to explain the (partly improper) name “Softmax”.

2.1.2 Problem 2: Softmax layer properties

- (a) Show/confirm that all outputs y_k from a softmax layer always are restricted to the unit interval $[0,1]$.
- (b) Show/confirm the K softmax outputs always sum to one, regardless of what are the inputs: $y_1 + y_2 + \dots + y_K = 1$.

2.2 Cross Entropy for classification

While the mean squared error is the standard cost function used to train ANNs when the response values (the “y -values”) are real numbers (regression), the standard cost function used for classification problems is the **Cross Entropy**. For two discrete probability distributions $\mathbf{p} = [p_1, p_2, \dots, p_d]^T$ and $\mathbf{q} = [q_1, q_2, \dots, q_d]^T$, which are specifying the probabilities p_i and q_i of d different possible outcomes, the Cross Entropy is defined as

$$H(\mathbf{p}, \mathbf{q}) = - \sum_{i=1}^d p_i \log(q_i) = \sum_{i=1}^d p_i \log(1/q_i)$$

For classification problems using a FANN with a Softmax layer at the end, $\mathbf{q}$ is the d-dimensional network output while $\mathbf{p}$ is a “one-hot-encoding” target vector that contains a 1 at a single unique position with the remaining elements being zero. For example, if $d = 4$ and the true class for an examples $\mathbf{x}_n$ is class 3, then $\mathbf{p}(n) = [0, 0, 1, 0]^T$.

Since by definition we have $0 \cdot \log(0) = 0$, the loss (Cross Entropy) for this example $\mathbf{x}_n$ will be $-\log(q_3)$. This means zero loss if $q_3 = 1$ and a very big loss when $q_3 \approx 0$. For a problem with N training examples the cost function $J()$ used for training of an ANN with its parameters collected in $\boldsymbol{\theta}$ is simply the average Cross Entropy across all examples:

$$J(\boldsymbol{\theta}) = -\frac{1}{N} \sum_{n=1}^N \sum_{i=1}^d p_{in} \log(q_{in}) = \frac{1}{N} \sum_{n=1}^N \sum_{i=1}^d p_{in} \log(1/q_{in})$$

2.2.1 Problem 3: Average Cross Entropy

Assume you have a 3-class classification problem where your ANN model produces the output $\mathbf{q}_1 = [0.01, 0.9, 0.09]^T$ for input example $\mathbf{x}_1$ and the output $\mathbf{q}_2 = [0.8, 0.15, 0.05]^T$ for input example $\mathbf{x}_2$. Determine the average Cross Entropy for this result if $\mathbf{x}_1$ belongs to class 2 and $\mathbf{x}_2$ belongs to class 1. Hint: This means that the desired output distributions for the two inputs $\mathbf{x}_1$ and $\mathbf{x}_2$ are $\mathbf{p}_1 = [0, 1, 0]$ and $\mathbf{p}_2 = [1, 0, 0]$, respectively. Answer: $-\log(\sqrt{0.72})$.

2.3 The Receiver Operating Characteristic (ROC) curve

For 2-class classification problems, one is often focusing on for example class 1 as the one of interest, while the second class corresponds to something uninteresting. For example, when trying to classify if a suitcase at the airport contains bombs or illegal drugs (class 1) or not (class 2) based on a vector $\mathbf{x}$ filled with sensor values, we are mainly interested in class 1. For such problems, it is natural to describe the performance of the classifier as a detector characterized by its probability of detection

$$P_D = P(\mathbf{x} \text{ is assigned to class 1} | \mathbf{x} \text{ belongs to class 1})$$

and its probability of false alarm P_{FA} .

$$P_{FA} = P(\mathbf{x} \text{ is assigned to class 1} | \mathbf{x} \text{ belongs to class 2})$$

Equivalent terminology

Often P_D is referred as one of the following:

- Probability of Detection P_D
- True Positive Rate (TPR)
- Sensitivity
- Recall

Similarly P_{FA} is often referred to as one of the following:

- Probability of False Alarm P_{FA}
- False Positive Rate (FPR)
- “one minus Specificity” (where Specificity s is defined as $s = 1 - P_{FA}$)

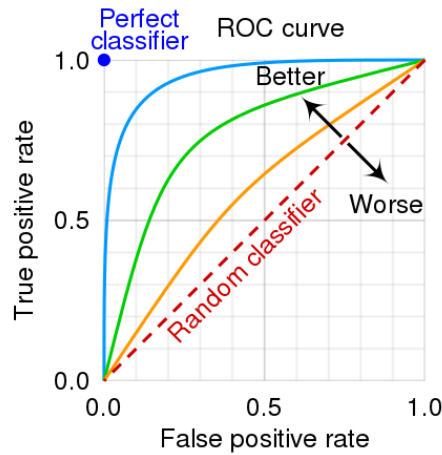


Figure 2: Examples of different Receiver Operating Characteristic (ROC) curves.

Changing detection threshold to get a ROC curve

Assume that we have built a FANN for a 2-class classification problem where the network has a softmax output layer consisting of two neurons, one for each class. Since the two outputs y_1 and y_2 always sum to one, $y_1 + y_2 = 1$, it is enough to consider for example y_1 for decision making as we always have $y_2 = 1 - y_1$. If we want to minimize the number of classification errors (meaning that missed detections and false alarms are equally costly), theory shows that the optimal decision is to choose class 1 if $y_1 > 0.5$. However, we can easily increase the number of examples assigned to class 1 by introducing an **adjustable threshold** τ smaller than 0.5, and then instead decide to choose class 1 as soon as $y_1 > \tau$. This means we will increase P_D while also increasing P_{FA} .

More generally, in the extreme case $\tau = -\infty$, all examples will be classified as class 1. This means $P_D = 1$ (very good) but at the same time $P_{FA} = 1$ so the classifier is useless. In the opposite extreme $\tau = \infty$, all examples will be classified as class 2. This means $P_{FA} = 0$ (very good), but at the same time $P_D = 0$ so the classifier is useless. By varying the threshold value continuously τ between these two extremes, we get the points $(P_{FA}(\tau), P_D(\tau))$ that are defining the so-called Receiver Operating Characteristic (ROC) curve. Some illustrative examples are shown in Fig. 2.

2.3.1 Problem 4: ROC curves

Consider the four different ROC curves illustrated in Fig. 2, each corresponding to a particular trained two-class classifier, for example FANNs having different architectures.

(a) Why should the classifier corresponding to the blue (top) curve be considered as the best performing classifier for the special case when the probability of false alarm is accepted to be 10%?

(b) By a “Random classifier” in this context we mean one which always (meaning without looking at the input $\mathbf{x}$) is choosing class 1 with some probability, for example 30%. What is P_D and P_{FA} in this case. Tip: See the corresponding ROC curve (line) in Fig. 2.

(c) Consider the two ROC curves A and B in Fig. 3, which have the same area under the curve (AUC), and they are crossing each other. If you can accept P_{FA} to be as high as 40%, which of the two classifiers A and B would you choose and why?

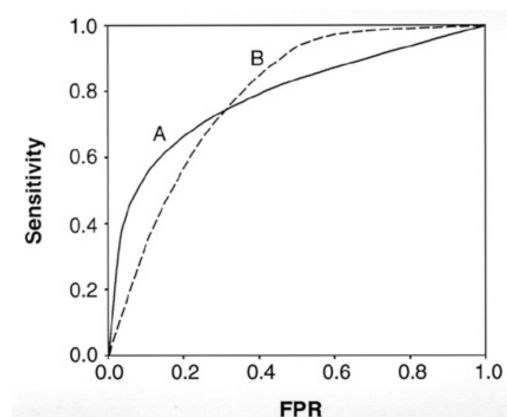


Figure 3: Examples of two crossing ROC curves with the same area under the curve (AUC).

2.4 Performance measures for more than two classes

For classification problems with more than two classes, the perhaps most common way of reporting the performance is the **accuracy** (probability of correct classification). However, this measure does not give much information about what types of errors are more common than others. In order to convey this information, in addition to the accuracy, one is often also providing a so-called confusion matrix. For one illustrative example of such a matrix, see as the one presented in Fig.4.

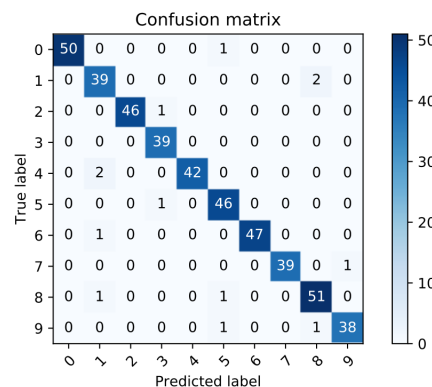


Figure 4: An example of a confusion matrix used to report the performance of a multi-class classifier.

Remark.

Confusion matrices are also often used for 2-class classification problems.

3 Task A: Two-class Signal Classification by FANNs

Classification of temporal signals (also known as time series) is often a challenging task, but with many potential applications, for example in speech/word recognition, music/song recognition, monitoring the mechanical sounds from engines and industrial machines, human heart monitoring, and stock market prediction.

Compared to previous regression problems considered, there are two main differences: (1) We are now switching from regression to classification. (2) We are switching from low-dimensional inputs to significantly higher dimensions and recognition tasks of the kind humans are good at. Thus we will be creating more spectacular forms of artificial intelligence.

3.1 Overall description

In this task, you are given a dataset containing a fairly large amount of “signal data”, where each temporal signal is labelled either as “normal” (numeric label 0) or “abnormal” (numeric label 1). Your task here is to construct a FANN that is able to achieve a high classification accuracy when trying to classify a signal as normal or abnormal. This means that this task is a 2-class classification problem. In Fig. 5 you can see examples of normal and abnormal signals. Notably, most of the code template used for performing regression in the previous assignment should be used here as well, after making necessary adjustments.

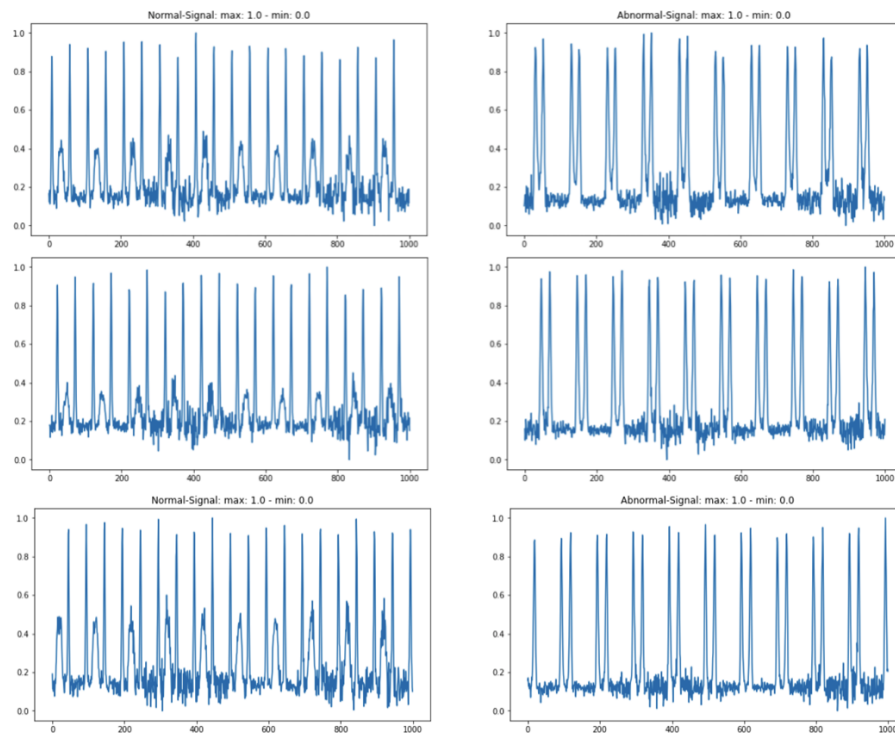


Figure 5: Examples of three “normal” (left column) and three “abnormal” (right column) signals.

3.2 Accessing provided data and code

You are provided with a zip-file on the relevant assignment page in Studium called `supporting_material_ann_for_classification.zip`.

It contains a folder called `1d_signal_data` containing the signal data of interest. It also contains a python file called `help_code_ann_for_classification.py`, which contains some code that you must import into your notebook in order to get access to the signal dataset as a PyTorch `Dataset`

object. Just make sure to have the folder and the python file in the same folder, next to each other. The following code shows how you can use `help_code_ann_for_classification.py`.

```
1 import help_code_ann_for_classification as help_code
2 signal_dataset = help_code.SignalDataset() # Provides back a PyTorch Dataset OBJECT!
3 size_of_signal_dataset = len(signal_dataset) # Print size of whole dataset
4
5 # Making a PyTorch DataLoader Object specifically for your Signal dataset that you got above.
6 import torch
7 import torch.nn as nn
8 from torch.utils.data import DataLoader
9 signal_dataloader = DataLoader(dataset=signal_dataset, batch_size=..., shuffle=True)
```

3.3 Problems

(a) Use the above code to get access to the signal dataset. Then explore it to find answers to the following: How many signals does the dataset contains and how long (in terms of time points) is each signal?

(b) Plot some of the signals in the dataset with their labels (normal/abnormal) shown in the plot title. Thus they should look like similar to the ones shown in Fig. 5.

(c) Split the original dataset into train-, validation- and test-sets. Choose some reasonable split values. You have to have enough data-points so that your network can learn as much and easily as possible while at the same time you also want enough data-points in your test set so that your estimate of the cost/accuracy on unseen data becomes as certain as possible. For example you may choose 70% for training, 20% for validation and 10% for test.

(d): Choose a first preliminary FANN architecture that seems suitable for this 2-class classification task. You will not need more than two hidden layers with with ReLU activation functions in all hidden nodes. Make sure that your network has the correct number of inputs to the first linear layer and that it has 2 final output nodes in the last linear layer.

IMPORTANT: Quite unexpectedly, and therefore also quite oddly, your network should **not** have any `nn.SoftMax()` layer as the last layer!

The reason for not having any Softmax layer is because the loss function `nn.CrossEntropyLoss()` actually employs the desired Softmax function itself to produce the network output during the calculation of the cost/loss! This means that the network you create will not have outputs that sum to one!

However, if you want to predict the actual class labels for a new input image, still you simply find the largest network output for each row and the corresponding class index, no softmax is needed. In terms of Python/PyTorch, you may apply `torch.argmax()` to the network output across the columns to get the index of the largest class prediction score for each row (data-point). Below you find a code snippet that helps you with this.

```
1 with torch.no_grad():
2     raw_y_preds = model(X) # predict model outputs for batch X
3
4 y_class_preds = raw_y_preds.argmax(dim=1) #compute class labels for each x_n
```

If a tensor `X` with the shape `[batch_size, input_dim]` contains set of signals as rows, then in the code above, `raw_y_preds` will be a tensor of shape `[batch_size, nr_classes]` where `nr_classes` denotes the number of classes. In our case `X` is a `[batch_size, 500]` tensor and `raw_y_preds` is a `[batch_size, 2]` tensor. The last row takes the raw outputs, and makes them into a `[batch_size, 1]` tensor, where each row now is the predicted class label for the corresponding signal. Notably, this is a general method for getting the predicted class labels from a classification ANN, and it works for any number of classes.

(e) Select and instantiate the loss function and the SGD optimizer with some suitable preliminary learning rate (the values 0.05 or 0.005 often work).

(f) As a first step, implement a very basic training loop consisting of only the mini-batch SGD inner-loop, without anything extra. Moreover, use only the set of training examples to check if you can get your model to reduce the cost/loss. This type of “test runs” of the training loop is always recommended as a sanity-check, to ensure that everything (model, batches and batch sizes) seem to work as expected. At this stage you just want to check if you can make the cost/loss on the mini-batches go down. This means you should monitor the mini-batch cost as a function of the number of mini-batches completed. When running multiple epochs, meaning the inner loop several times, you should aim for results in a graph similar to the one in Fig. 6.

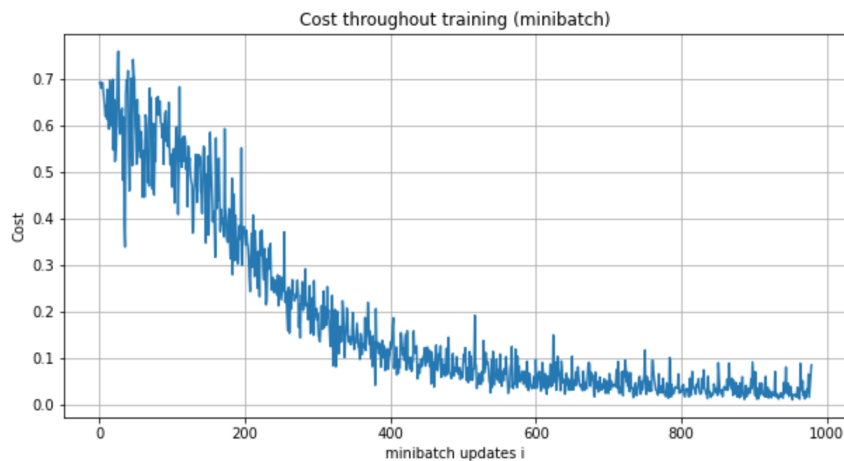


Figure 6: Example of how the cost is decreasing as a function of the number of mini-batches performed. Note: This is the result of running several epochs.

If you can do this you are in good progress. This means you can go on and try to implement the complete training loop, including model performance evaluation using both the training and validation sets.

(g) Define the function for model performance evaluation based on the template. Remember, this is classification task, and as such we should be interested in the accuracy of our model.

(h) Integrate your model performance evaluation function into your training loop and perform model evaluation on both validation and train sets for every epoch, before the inner SGD training loop. Keep track and store these performance metrics during your whole training session, because you will need to plot and report these values for your fully trained network. Your training should result in plots looking similar to those in Fig. 7 and Fig. 8. Your training runs should look something similar to Fig. 9 if you print the performance metrics accordingly.

(i) From the results obtained showing the results on the training and validation sets in the same graph, do you see any signs of overfitting?

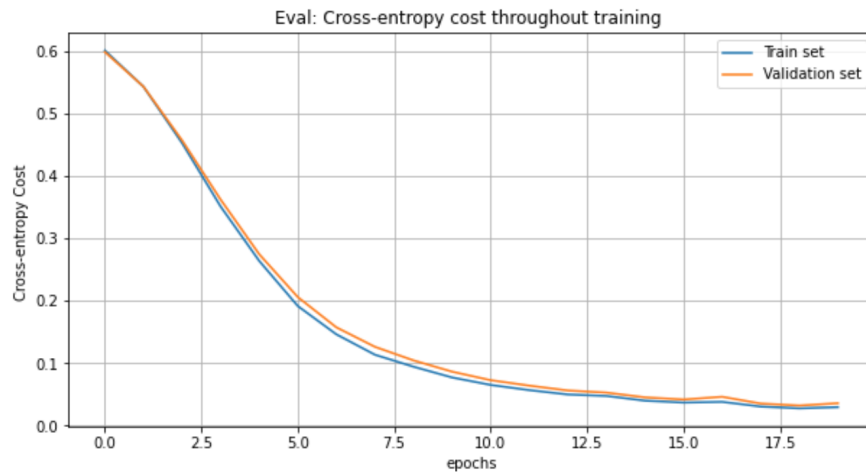


Figure 7: Example of how the average Cross Entropy is developing as a function of the number of epochs performed.

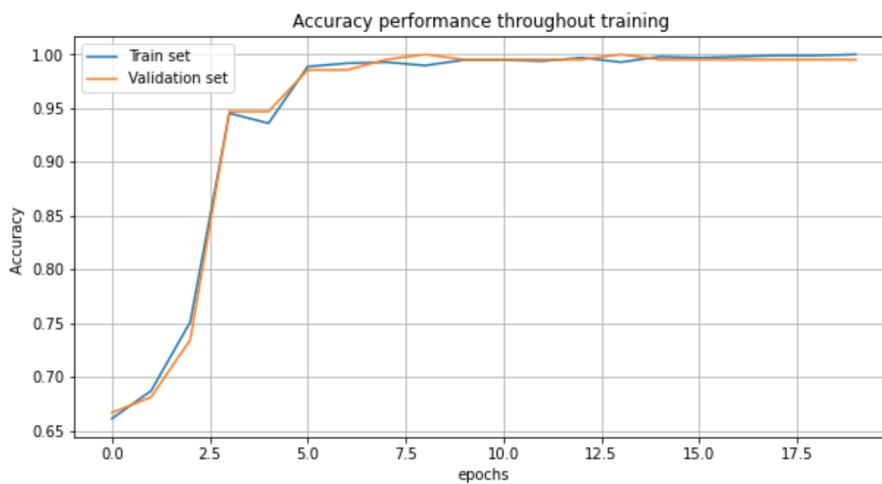


Figure 8: Example of how the accuracy is developing as a function of the number of epochs completed.

```
Epoch: 0 - Train cost: 0.5982961058616638, - Train Acc: 0.6621900826446281
Epoch: 0 - Val cost: 0.5973932147026062, - Val Acc: 0.6666666666666666
.....

Epoch: 1 - Train cost: 0.5307530164718628, - Train Acc: 0.6621900826446281
Epoch: 1 - Val cost: 0.5302639603614807, - Val Acc: 0.6666666666666666
.....

Epoch: 2 - Train cost: 0.44418027997016907, - Train Acc: 0.8584710743801653
Epoch: 2 - Val cost: 0.44962990283966064, - Val Acc: 0.855072463768116
.....

Epoch: 3 - Train cost: 0.33845898509025574, - Train Acc: 0.9132231404958677
Epoch: 3 - Val cost: 0.3449340760707855, - Val Acc: 0.9178743961352657
.....

Epoch: 4 - Train cost: 0.2543441951274872, - Train Acc: 0.9586776859504132
Epoch: 4 - Val cost: 0.26432108879089355, - Val Acc: 0.961352657004831
.....

Epoch: 5 - Train cost: 0.18789392709732056, - Train Acc: 0.984504132231405
Epoch: 5 - Val cost: 0.2002243995666504, - Val Acc: 0.9806763285024155
.....
```

Figure 9: Example of performance metrics printouts during a training session.

4 Task B: Multi-class Image Classification of FashionMNIST by FANNs

In this task we are switching from classification of 1D signals to classification of 2D images. Here you will train a FANN to classify images of clothing items, into 10 different clothing categories. The steps involved here are more or less identical to the signal classification task previously. The key differences are: (1) The dataset contains 2D images instead of 1D signals. (2) The number of classes are 10 instead of 2. (3) The required FANN architecture is different and the pixel values corresponding to each 2D image must be “flattened” into a numerical vector.

4.1 The images

FashionMNIST¹ is a dataset of article images from the company Zalando. It consists of 70000 images of clothing items in gray-scale, divided into one training set consisting of 60000 examples and one test set consisting of 10000 examples. Each example is a 28×28 grayscale image and comes with a label corresponding to one out of 10 classes. For illustrative examples of the dataset content, see Fig. 10.

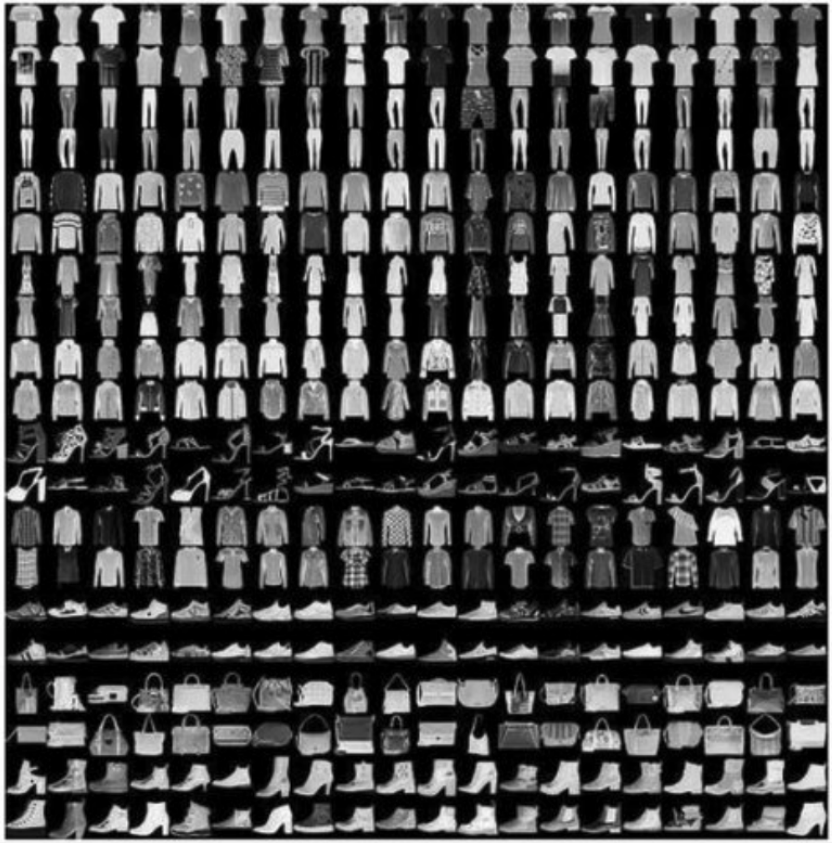
Label	Description	Examples
0	T-Shirt/Top	
1	Trouser	
2	Pullover	
3	Dress	
4	Coat	
5	Sandals	
6	Shirt	
7	Sneaker	
8	Bag	
9	Ankle boots	

Figure 10: Illustration of the FashionMNIST dataset showing samples from each of the 10 clothing classes.

¹Zalando intends Fashion-MNIST to serve as a direct replacement for the original MNIST dataset for benchmarking machine learning based classification algorithms. The original MNIST dataset contains handwritten digits and is very popular within the AI/ML/Data Science community, where it is often used as a benchmark to validate and compare different algorithms.

4.2 The labels

From Fig.10 we can see that each training and test example is assigned to one of the following 10 labels:

- 0-T-shirt/top
- 1-Trouser
- 2-Pullover
- 3-Dress
- 4-Coat
- 5-Sandal
- 6-Shirt
- 7-Sneaker
- 8-Bag
- 9-Ankle boot

Remark.

For more details if you are interested, see the following links.

<https://www.kaggle.com/zalando-research/fashionmnist>

<https://github.com/zalandoresearch/fashion-mnist>

4.3 Importing the dataset

As shown below, importing the dataset is conveniently handled by PyTorch since this is an established standard dataset. As a matter of fact, we have already shown this code (in the previous assignment on regression).

```
1 ## Code to access the FashionMNIST
2 import torch
3 from torch import nn
4 from torch.utils.data import DataLoader
5 from torchvision import datasets
6
7 from torchvision.transforms import ToTensor, Lambda, Compose
8 import matplotlib.pyplot as plt
9
10 # Download training data from open datasets.
11 training_data = datasets.FashionMNIST(
12     root="data",
13     train=True,
14     download=True,
15     transform=ToTensor(),
16 )
17
18 # Download test data from open datasets.
19 test_data = datasets.FashionMNIST(
20     root="data",
21     train=False,
22     download=True,
23     transform=ToTensor(),
24 )
```

Notably, the data are already “pre-tensored”: from here on you need to only provide this dataset object to a `DataLoader` object to use in your training loop.

4.4 The actual tasks to perform

(a) Repeat the procedure you performed when designing and testing the signal classifiers in Task A, this time using the FashionMNIST dataset. The main difference here:

1. Images that need to be flattened into vectors (each having $28 \cdot 28 = 784$ elements) before they can be presented to the FANN. How we deal with this is by placing a `nn.Flatten()` component at the beginning of our architecture specification in `nn.Sequential(...)`. Tip: See “But what is a neural network? — Chapter 1, Deep learning” from 3Blue1Brown provided in the Extra materials at the end of this document.
2. A 10-class problem instead of a 2-class problem.

(b) Determine and print out the accuracy of the final FANN classifier based on the examples in the independent test set.

(c) Based on your own interests and perhaps some internet search: Propose at least two concrete potential areas of industrial or research applications where you personally think image classification of this kind can be (or already is) of practical and commercial importance.

5 Task C: Convolutional ANNs

Although fully connected FANNs can be quite powerful for both regression and classification problems, when it comes to processing and analyzing images and temporal data, there are multiple reasons for using *convolutional ANNs* (CNNs) instead. An example CNN architecture is presented in Fig. 11. Three of the most important reasons for using a CNN instead of a fully connected FANN in for structured data like images, time-series and audio-files are:

- Much fewer total number of parameters to tune for the same number of neurons.
- Much faster computations because they are mainly mathematical convolutions that can be rapidly calculated using dedicated hardware and the Fast Fourier Transform algorithm.
- Prior knowledge/assumptions are build into the FANN network about the problem task. In particular, the networks are often designed to create a hierarchy of so called “feature maps” where each feature map is designed to pick up particular input features that may occur at any location in the image, and that are nontrivial to discover/engineer manually.

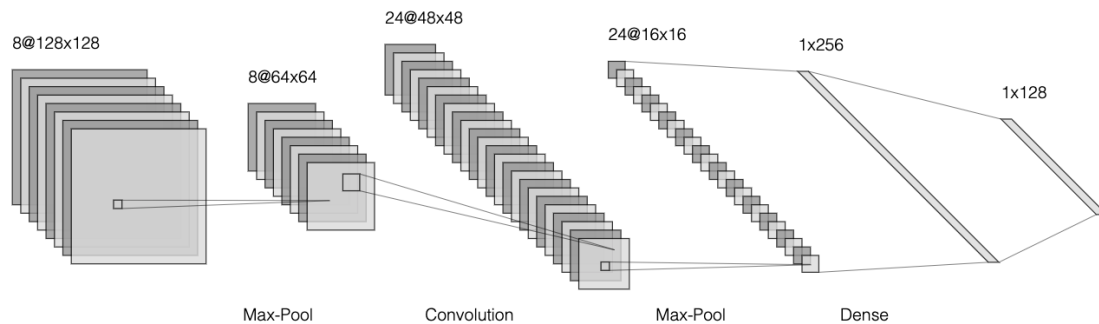


Figure 11: An example of a convolutional ANN.

5.1 Problem 1: CNN Theory

TIP:To find good answers to this problem, watch the video linked below.
(Justin Johnson’s CNN lecture at U. Michigan):

<https://www.youtube.com/watch?v=ANyxBVxmdZ0>

Consider a single CNN features map produced directly from an input image. Assume that the size of the kernel is $m \times m$. This corresponds to each neuron having a receptive field that is limited to a $m \times m$ part of the input image.

- (a) How many adjustable parameters does the feature map corresponding to this kernel have?
- (b) Assume your CNN has 2 feature maps produced directly from the input image, all with kernel size $m \times m$. How many adjustable parameters are there in this case? Why does the number of adjustable parameters not depend on the size of the input image?
- (c) Assume you have small images of size 28×28 pixels. What are reasonable sizes of the kernel used in a CNN that should process this image? In particular, why is for example the kernel size 27×27 unreasonable large here? Tip: If you want to know more about the convolution operator, have a look at the 20 minutes video provided via this link <https://youtu.be/B-M5q51U8SM>. This is to understand more about how convolution works in 2D in terms of a reversed (rotated) kernel, and in the context of ANNs.

5.2 Problem 2: Image Classification using a CNN

- (a) Run the following code as a whole or in parts. Make sure you can explain what it does on each row.
- (b) What is the size of the kernel (filter) in the first layer of the CNN implemented by this code?
- (c) Explain by means of words, numbers, or pictures what is the structure of the CNN implemented by this code?
- (d) Change the number of epochs to 10 and start a longer run. Be patient when waiting for the code to finish!

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torchvision
5 import torchvision.transforms as transforms
6
7 # Define the CNN model
8 class Net(nn.Module):
9     def __init__(self):
10         super(Net, self).__init__()
11         self.conv1 = nn.Conv2d(1, 32, 3)
12         self.pool = nn.MaxPool2d(2, 2)
13         self.conv2 = nn.Conv2d(32, 64, 3)
14         self.fc1 = nn.Linear(64 * 5 * 5, 128)
15         self.fc2 = nn.Linear(128, 10)
16
17     def forward(self, x):
18         x = self.pool(torch.relu(self.conv1(x)))
19         x = self.pool(torch.relu(self.conv2(x)))
20         x = x.view(-1, 64 * 5 * 5) # PyTorch's equivalent to numpy.reshape()
21         x = torch.relu(self.fc1(x))
22         x = self.fc2(x) #No softmax at the end although classification
23         return x
24
25 # Initialize the model
26 net = Net()
27
28 # Define the loss function and optimizer
29 criterion = nn.CrossEntropyLoss()
30 optimizer = optim.Adam(net.parameters(), lr=0.001)
31
32 # Load and preprocess the FashionMNIST dataset
33 transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5,), (0.5,))])
34 trainset = torchvision.datasets.FashionMNIST(root='./data', train=True, download=True, transform=
    transform)
35 trainloader = torch.utils.data.DataLoader(trainset, batch_size=64, shuffle=True)
36
37 # Training loop
38 for epoch in range(1): # Change the number of epochs as needed
39     running_loss = 0.0
40     for i, data in enumerate(trainloader, 0):
41         inputs, labels = data
42
43         optimizer.zero_grad()
44
45         outputs = net(inputs)
46         loss = criterion(outputs, labels)
47         loss.backward()
48         optimizer.step()
49
50         running_loss += loss.item()
51         if i % 100 == 99: # Print every 100 mini-batches
52             print(f'[Epoch {epoch + 1}, Batch {i + 1}] Loss: {running_loss / 100:.3f}')
53             running_loss = 0.0
54
55 print('Finished Training')
56
57 # Save the parameters of the trained model
58 torch.save(net.state_dict(), 'fashion_mnist_cnn.pth')
```

- (e) After completing the training above, run the following code to determine the accuracy (probability of correct classifications) of the final trained classifier on the training set itself. What is the loss/cost function used here, and why is not probability of missclassification used as a cost function instead?

```
1 #Perform the testing batchwise
2 correct_train= 0
3 total_train = 0
4 with torch.no_grad():
5     for images, labels in trainloader:
6         outputs = net(images)
7         predicted_values, predicted_indicies = torch.max(outputs.data, 1)
8         total_train+= labels.size(0)
9         correct_train += (predicted_indicies == labels).sum().item()
10
11
12 #Calculate Accuracy
13 accuracy_train = 100 * correct_train / total_train
14 print(f'Accuracy on the train dataset: {accuracy_train:.2f}%')
15 print("No of training examples:", total_train)
```

5.3 Problem 3: Performance Estimation of a CNN

(a) After completing the above problems, estimate the true performance of your trained CNN classifier by using the following code. It imports the saved weights of a trained CNN and tests it on the examples in the independent test set.

```
1 loaded_model = Net() #initializa the model
2 loaded_model.load_state_dict(torch.load('fashion_mnist_cnn.pth'))
3 loaded_model.eval() #Put it in evaluation mode
4
5
6 # Load and preprocess the FashionMNIST TEST dataset
7 transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5,), (0.5,))])
8 test_dataset = torchvision.datasets.FashionMNIST(root='./data', train=False, transform=transform,
9         download=True)
10 test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=64, shuffle=False)
11
12 #Perform the testing batchwise
13 correct_test = 0
14 total_test = 0
15 with torch.no_grad():
16     for images, labels in test_loader:
17         outputs = loaded_model(images)
18         predicted_values, predicted_indicies = torch.max(outputs.data, 1)
19         total_test += labels.size(0)
20         correct_test += (predicted_indicies == labels).sum().item()
21
22 #Calculate Accuracy
23 accuracy_test = 100 * correct_test / total_test
24 print(f'Accuracy on the test dataset: {accuracy_test:.2f}%')
25 print("No of testexamples:", total_test)
```

(b) Make a comparison - any big difference between the accuracy estimates obtained with the test and training sets? What is your interpretation of this result in terms of overfitting?

(c) Compare your accuracy on the external test set obtained here with the corresponding accuracy obtained using the FANN studied earlier using the same test set. Which of the networks seems to perform best?

6 DEADLINE AND EXAMINATION

This examination will consist of the following parts:

(1) Answers and Results: Uploading answers and results to all mandatory problems present above to Studium. **Deadline: See course information..**

NOTE: If you are working in pairs, each person has to submit their own document, even if it is almost identical. You are of course always responsible for your own submission and must be able to explain all of it during seminars and similar examinations.

(2) Python code as one notebook or as wrapped inside a zip file: The code should be ready to run in Google Colab. If your code consists of more than one notebook (and/or some additional code called from inside the notebook) your instructions how to run the code should be provided as a text file called `readme.txt`. Make sure that all files you are submitting have your first name and surname as part of the file name, for example like:

`Assignment_3_reflections_Mats_Gustafsson.pdf`

NOTE: If you are working in pairs, each person has to submit their own code, even if it is almost identical. Notably, you are of course responsible for all your the code your submit, and you must be prepared to discuss and explain it during the seminars.

(3) Personal reflections and feedback: An uploaded pdf document to Studium providing personal reflection and feedback: This pdf document should contain reflective answers to the following questions:

- (a) What are the main differences between training a FANN for a regression problem compared to a classification problem - in terms of network structure and cost function?
- (b) Why is it good result when a 2-class classifier has a large AUC (area under the ROC curve)?
- (c) Why is it good/promising when you can see that the accuracy for the training and test set are following each other during/after training?
- (d) What did you personally find interesting and/or useful an/or informative with the assignment, and why? If you actually did not find it enlightening, in that case we would like to know about why.
- (e) What did you find frustrating and/or difficult, and/or unnecessary, and why?
- (f) What would you suggest as an improvement of this assignment for next year, and why?
- (g) Did you spend more than 20 hours to complete this exercise? Why?

NOTE: If you are working in pairs, each person has to send in a personal reflection.

(4) Mandatory Seminar: You discussing/presenting your results during the associated mandatory seminar. You do NOT have to prepare any separate slide presentation, it will be fine to show your Colab notebook if/when you are requested to present something.

Good Luck and Enjoy!

7 Extra Study Material, Tips, Etc

7.1 PyTorch Help, Troubleshooting and Guidance

1. A gentle introduction to training ANNs In PyTorch: Training a classification model on MNIST with PyTorch <https://www.youtube.com/watch?v=OMDn66kM9Qc>
2. Certain bugs and oddities are more common than others. Here is a link to a 20 minutes long video that gives you 10 great practical tips when you are about to start training your PyTorch models <https://www.youtube.com/watch?v=02wJ3tkc-TU>

7.2 Neural Networks, what they actually do and how they do it

1. “But what is a neural network? — Chapter 1, Deep learning” from 3Blue1Brown - https://www.youtube.com/watch?v=aircAruvnKk&list=PLZHq0b0WTQDNU6R1_67000Dx_ZCJB-3pi&index=1

7.3 Gradient Decent and Stochastic Gradient Decent

1. “Gradient descent, how neural networks learn — Chapter 2, Deep learning” from 3Blue1Brown - https://www.youtube.com/watch?v=IHZwWfHwA-w&list=PLZHq0b0WTQDNU6R1_67000Dx_ZCJB-3pi&index=2
2. “Stochastic Gradient Descent, Clearly Explained!!!” by StatQuest - <https://www.youtube.com/watch?v=vMh0zPT0tLI>
3. “25. Stochastic Gradient Descent” Suvrit Sra from MIT 18.065 Matrix Methods in Data Analysis, Signal Processing, and Machine Learning, Spring 2018 - <https://www.youtube.com/watch?v=k3AiUhwHQ28>
4. “Lecture 17.2 — Large Scale Machine Learning — Stochastic Gradient Descent” by Andrew Ng - <https://www.youtube.com/watch?v=W9iWNJNFzQI>
5. “The Stochastic Gradient Descent Algorithm” by Nathan Kutz https://www.youtube.com/watch?v=_adhFSH66jc

7.4 Backpropagation to achieve auto-differentiation

1. “What is backpropagation really doing? — Chapter 3, Deep learning” from 3Blue1Brown - https://www.youtube.com/watch?v=Ilg3gGewQ5U&list=PLZHq0b0WTQDNU6R1_67000Dx_ZCJB-3pi&index=3
2. “Backpropagation calculus — Chapter 4, Deep learning” from 3Blue1Brown - https://www.youtube.com/watch?v=tIeHLnjs5U8&list=PLZHq0b0WTQDNU6R1_67000Dx_ZCJB-3pi&index=4

7.5 Convolutional Neural Networks

1. “Lecture 7: Convolutional Neural Networks” by Andrej Karpathy from Stanford’s CS231n Winter 2016 [de facto canonical lecture on CNNs] - <https://www.youtube.com/watch?v=LxfUGhug-iQ>
2. “A Comprehensive Guide to Convolutional Neural Networks — the ELI5 way” by Sumit Saha - <https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53>